
Revisiting Distribution Correction Estimation for Offline Imitation Learning with Suboptimal Dataset

Quang Anh Pham¹ Tien Mai¹ Akshat Kumar¹

Abstract

Imitation Learning (IL) learns high-quality policies from expert demonstrations but degrades in low-expert-data regimes. To address this, recent work studies “*offline IL with supplementary data*”, augmenting expert data with trajectories from sub-optimal policies. A prominent framework is Distribution Correction Estimation (DICE), which estimates density ratios via the dual of a divergence minimization problem between learned and expert visitation distributions. However, existing DICE-based methods either rely on a strict coverage assumption or introduce additional dataset regularization, limiting performance. We propose **ReDICE**, a new DICE-based method that addresses these issues through an objective-level reformulation. Our approach constructs a mixture-distribution objective that preserves the original expert-imitation objective while removing the coverage assumption, and its dual reduces to a stable Gumbel regression objective for efficient optimization. We further introduce a novel policy extraction mechanism that improves performance. Experiments on standard and real-world offline IL benchmarks show that ReDICE consistently outperforms prior methods and achieves state-of-the-art results.

1. Introduction

Offline IL Learning (IL) aims to learn decision-making policies by mimicking expert behavior from demonstrations, without requiring explicit reward specification or online interaction with the environment (Zare et al., 2024). This paradigm has proven effective across a wide range of sequential decision-making tasks, particularly in safety-critical

domains such as robotics and autonomous driving, where exploration is expensive or infeasible (Cheng et al., 2024; Black et al., 2025). The most widely used IL method is Behavior Cloning (BC) (Pomerleau, 1988), which reduces policy learning to supervised learning on expert state–action pairs. Despite its simplicity, BC is fundamentally limited by compounding errors (Ross et al., 2011), where minor execution deviations can lead the agent into unseen states from which it may not recover. To overcome this limitation, recent works (Xu et al., 2022; Li et al., 2023b) have studied *offline IL with suboptimal demonstrations* setting in which a small set of expert demonstrations is supplemented with a much larger dataset collected by suboptimal behavior policies. Such supplementary data are often inexpensive to obtain and can significantly expand state–action coverage. However, directly leveraging these data remains challenging, since naively imitating suboptimal behaviors can degrade policy performance.

A prominent class of methods addressing this setting is Distribution Correction Estimation (DICE) (Nachum et al., 2019), which formulates imitation learning as the minimization of a divergence between the occupancy measures of the learned policy and expert. Despite their strong theoretical foundations, existing DICE formulations suffer from important practical and conceptual limitations. The first DICE method, DemoDICE (Kim et al., 2021) modifies the original divergence objective by introducing an additional KL regularization term that penalizes deviation from the suboptimal data distribution. Although this regularization helps integrate suboptimal data into the learning process, it comes at the cost of no longer directly minimizing the divergence between the learned policy and the expert. Moreover, the strength of the regularization must be carefully tuned, and overly conservative regularization can prevent effective policy improvement when supplementary data are low-quality (Yu et al., 2023). Another approach, SMODICE (Ma et al., 2022) directly minimizes the KL divergence between the learned policy occupancy and the expert occupancy by decomposing the divergence using the suboptimal data distribution as a reference measure. However, this decomposition is only valid under a strict coverage assumption—the suboptimal dataset assigns non-zero probability to all state–action pairs visited by the expert. In practice, violations of this

¹Singapore Management University, Singapore. Correspondence to: Quang Anh Pham <qa.pham.2025@phdcs.smu.edu.sg>.

assumption can often lead to unstable optimization and poor performance (Sikchi et al., 2024).

These observations reveal a fundamental limitation of existing DICE-based methods: enforcing expert coverage through suboptimal data can lead to biased or unstable objectives, while adding regularization compromises the original imitation learning objective. This raises a natural question: **Is it possible to design a DICE objective that remains well-defined without coverage assumptions while still directly optimizing expert imitation?** We answer this question in this paper by proposing ReDICE (**R**evisiting **D**istribution **C**orrection **E**stimation), a new offline IL learning method that resolves these issues. Specifically, our contributions are as follows:

- We propose a mixture-based KL divergence formulation for offline IL, which directly minimizes the divergence between the learned policy occupancy and the expert occupancy. This formulation removes the strict expert coverage assumption and avoids introducing auxiliary regularization terms, while preserving a principled divergence minimization objective.
- We derive a dual representation of the proposed objective via Lagrangian relaxation of Bellman flow constraints and show that the resulting dual objective is equivalent to a Gumbel regression loss (Garg et al., 2023). This loss is strictly convex with a unique global optimum that helps stabilize the learning process and improve convergence. Building on this dual formulation, we develop a practical policy extraction mechanism that mitigates weight collapse caused by value overestimation, while remaining consistent with the theoretical solution.
- We evaluate ReDICE through extensive experiments on standard offline IL benchmarks with different levels of expert coverage, and a real-world maritime navigation setting. Experimental results show that ReDICE outperforms state-of-the-art baselines. The implementation of ReDICE is publicly available at <https://github.com/quanganh1999/ReDICE>.

2. Related Work

Offline IL with suboptimal data Learning solely from expert demonstrations often requires a large amount of expert data, which can be costly or difficult to obtain in practice. To address this, prior work has studied offline imitation learning with suboptimal data, where limited expert demonstrations are augmented with inexpensive but imperfect trajectories. DemoDICE (Kim et al., 2021) extends the BC objective by introducing a KL-divergence regularization term that constrains the learned policy to remain close to the behavior policy. However, its performance can degrade when the supplementary data are highly suboptimal. Subsequent approaches mitigate this issue by proposing different

variants of weighted BC. DWBC (Xu et al., 2022) learns a discriminator to assign importance weights to demonstrations, while ISWBC (Li et al., 2023b) reweights the BC loss using a density ratio between expert and offline data to prioritize expert-like samples. ILID (Yue et al., 2024) further leverages environment dynamics to identify beneficial behaviors that lead the agent back to known expert states, thereby empowering the policy to recover from deviations using imperfect but informative demonstrations. SMODICE (Ma et al., 2022) considers a related setting in which the expert dataset contains only states without actions. Unlike DemoDICE, SMODICE directly minimizes the KL divergence between the learned policy occupancy and the expert occupancy using offline data. While theoretically appealing, this approach relies on a restrictive coverage assumption that the suboptimal data cover expert visitations. ReCOIL (Sikchi et al., 2024) relaxes this assumption by optimizing alternative divergence minimization between the two mixture occupancy measures. While this modification improves performance in practice, ReCOIL does not guarantee recovery of the optimal density ratio corresponding to the original expert-policy divergence objective. We provide a detailed comparison between ReCOIL and our formulation in Appendix B, showing that ReDICE preserves equivalence to the original KL-based imitation objective while avoiding the max-min optimization and additional surrogate used in ReCOIL.

DICE-based Methods Stationary distribution correction estimation was first introduced by (Nachum et al., 2019) as the DICE framework, which enables estimation of the stationary distribution of a target policy using off-policy data. The key idea is to formulate learning objectives directly in terms of distribution ratios through duality and change-of-variables via importance sampling (Nachum & Dai, 2020). This framework has since been applied successfully across a variety of reinforcement learning (RL) problems: off-policy evaluation (Zhang et al., 2020; Yang et al., 2020), reward learning (Li et al., 2023a), offline RL (Lee et al., 2021; Mao et al., 2024), constrained RL (Lee et al., 2022), unsupervised RL (Lee et al., 2025), online IL (Kostrikov et al., 2020), and offline IL (Kim et al., 2021; Ma et al., 2022). ReDICE differs from prior DICE-based offline IL methods by using supplementary data as a mixture reference rather than as the direct imitation target or only as a behavior regularizer. This design relaxes the coverage limitations of expert-only matching while remaining equivalent to the original KL-based imitation objective.

3. Preliminaries

We consider a Markov Decision Process (MDP) denoted by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, d_0)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s'|s, a)$ is the transition

dynamics, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, $\gamma \in (0, 1)$ is the discount factor, and $d_0(s)$ is the initial state distribution. A stochastic policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps each state to a distribution over actions. Executing the policy π in MDP $\mathcal{M}$ induces a discounted state-action visitation distribution, known as the occupancy measure:

$$d^\pi(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \Pr(s_t = s, a_t = a \mid s_0 \sim d_0, a_t \sim \pi(\cdot \mid s_t), s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)). \quad (1)$$

We can represent the standard RL objective which finds a policy maximizing the expected return under its occupancy measure i.e. $\max_{\pi} \{\mathbb{E}_{s,a \sim d^\pi} [r(s, a)]\}$.

3.1. Offline RL with Distribution Correction Estimation

Assuming that we have access to a static dataset $\mathcal{D} = (s, a, r, s')$ generated by any behavior policy. The goal of DICE approaches (Nachum & Dai, 2020) is leveraging $\mathcal{D}$ to optimize $\max_{\pi} \{\mathbb{E}_{s,a \sim d^\pi} [r(s, a)]\}$, thus removing the need of on-policy samples from d^π . Specifically, they add a behavior regularization term into the original objective as:

$$\max_{\pi} \mathbb{E}_{s,a \sim d^\pi} [r(s, a)] - \alpha D_f(d^\pi(s, a) \parallel d^D(s, a)) \quad (2)$$

where D_f is an f -divergence between two distributions, and $d^D(s, a)$ denotes the visitation distribution of policy generating dataset $\mathcal{D}$. We then omit d^π from (2) by rewriting it as an optimization problem over the space of valid state-action visitation distributions:

$$\max_{d \geq 0} \mathbb{E}_{s,a \sim d} [r(s, a)] - \alpha D_f(d(s, a) \parallel d^D(s, a)) \quad (3)$$

$$\text{s.t.} \quad \sum_a d(s, a) = (1 - \gamma) d_0(s) + \gamma \mathcal{T}_* d(s), \quad \forall s \quad (4)$$

where $\mathcal{T}_* d(s) = \sum_{\bar{s}, \bar{a}} d(\bar{s}, \bar{a}) \mathcal{P}(s \mid \bar{s}, \bar{a})$, and (4) are *Bellman flow constraints* (Manné, 1960) ensuring that $d(s, a)$ is a valid occupancy measure.

Solving the primal problem in Eq. (3) directly is intractable due to the flow constraints. We convert it to an unconstrained formulation by applying Lagrangian duality (Detailed Derivation in Appendix A.1):

$$\min_V (1 - \gamma) \mathbb{E}_{s \sim d_0} [V(s)] + \alpha \mathbb{E}_{s,a \sim d^D} [f^* \left(\frac{\mathcal{T}V(s, a) - V(s)}{\alpha} \right)] \quad (5)$$

where $f^*(x)$ is a variant of convex conjugate defined as $f^*(x) = \max\{0, (f')^{-1}(x)\} \cdot x - f(\max\{0, (f')^{-1}(x)\})$, and $\mathcal{T}V(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot \mid s, a)} [V(s')]$ is a Bellman operator on V . The optimal solution V^* of (5) can be used to compute the optimal density ratio (distribution correction):

$$\frac{d^*(s, a)}{d^D(s, a)} = \max \left(0, (f')^{-1} \left(\frac{\mathcal{T}V(s, a) - V(s)}{\alpha} \right) \right) \quad (6)$$

3.2. DICE for Offline IL with Supplementary Data

In this setting, we are provided with two static datasets: an expert dataset $\mathcal{D}_E = \{(s, a, s')\}$ generated by an expert policy π_E , and a suboptimal dataset $\mathcal{D}_S = \{(s, a, s')\}$ generated by low-quality policies. We denote the stationary visitation distributions of these datasets as $d^E(s, a)$ and $d^S(s, a)$, respectively. The goal is to learn a policy π that mimics the expert behavior using only $\mathcal{D}_E$ and $\mathcal{D}_S$, without environment interaction. We use the shorthand $D_{\text{KL}}(p \parallel q)$ for $D_{\text{KL}}(p(s, a) \parallel q(s, a))$ with some state-action visitation distributions p and q to simplify the notation. We consider two main DICE formulations introduced in previous works for the Offline IL with Supplementary Data: SMODICE (Ma et al., 2022) and DemoDICE (Kim et al., 2021).

Even though SMODICE focuses on minimizing the KL divergence between two state visitation distributions of policy and expert, we can easily extend its formulation to state-action visitation distribution as:

$$D_{\text{KL}}(d^\pi(s, a) \parallel d^E(s, a)) \quad (7)$$

$$= \mathbb{E}_{d^\pi} \left[\log \left(\frac{d^\pi(s, a)}{d^E(s, a)} \cdot \frac{d^S(s, a)}{d^S(s, a)} \right) \right] \quad (8)$$

$$= \mathbb{E}_{d^\pi} \left[\log \frac{d^S(s, a)}{d^E(s, a)} \right] + \mathbb{E}_{d^\pi} \left[\log \frac{d^\pi(s, a)}{d^S(s, a)} \right] \quad (9)$$

$$= \mathbb{E}_{d^\pi} \left[\log \frac{d^S(s, a)}{d^E(s, a)} \right] + D_{\text{KL}}(d \parallel d^S) \quad (10)$$

SMODICE relies on the coverage assumption ($d^S(s, a) > 0$ whenever $d^E(s, a) > 0$) to ensure that the log decomposition (Step (8) to (9)) is valid. Based on (10), we can convert $\min_{\pi} D_{\text{KL}}(d^\pi(s, a) \parallel d^E(s, a))$ into the following maximization problem:

$$\max_{\pi} \mathbb{E}_{d^\pi} \left[-\log \frac{d^S(s, a)}{d^E(s, a)} \right] - D_{\text{KL}}(d \parallel d^S) \quad (11)$$

(11) is exactly a DICE objective (2) if we set $r(s, a) = -\log \frac{d^S(s, a)}{d^E(s, a)}$, $\alpha = 1$, and $d^D = d^S$. To estimate $r(s, a)$, we train a discriminator $c : \mathcal{S} \times \mathcal{A} \rightarrow (0, 1)$ using GAN objective (Goodfellow et al., 2014):

$$\max_c \mathbb{E}_{d^E} [\log c(s, a)] + \mathbb{E}_{d^S} [\log(1 - c(s, a))] \quad (12)$$

The optimal discriminator $c^*(s, a) = \frac{d^E(s, a)}{d^E(s, a) + d^S(s, a)}$. Therefore, we can compute $r(s, a) = -\log \left(\frac{1}{c^*(s, a)} - 1 \right)$

DemoDICE modifies the original divergence objective by introducing an additional KL regularization term that penalizes deviation from the suboptimal data distribution:

$$\min_{\pi} D_{\text{KL}}(d^\pi \parallel d^E) + \lambda D_{\text{KL}}(d^\pi \parallel d^S) \quad (13)$$

Similar to SMODICE, they require the coverage assumption to decompose $D_{KL}(d^\pi(s, a) \parallel d^E(s, a))$ to obtain a DICE objective:

$$\max_{\pi} \mathbb{E}_{d^\pi} \left[-\log \frac{d^S(s, a)}{d^E(s, a)} \right] - (1 + \lambda) D_{KL}(d \parallel d^S) \quad (14)$$

We can see that the primary limitation of both SMODICE and DemoDICE is reliance on the coverage assumption. DemoDICE does not directly minimize expert-policy divergence and depends on a KL regularization term controlled by the hyperparameter λ .

4. Methodology

In this section, we present ReDICE, a novel offline IL method to deal with the coverage-assumption issue of previous DICE works while still directly optimizing the KL divergence between policy and expert.

4.1. Relaxing Coverage Assumption via Mixture Distribution

Different from prior works, we decompose the KL divergence $D_{KL}(d^\pi \parallel d^E)$ using a mixture distribution $d^{mix} = \beta d^E + (1 - \beta) d^S$ with $\beta \in (0, 1)$ as follows:

$$D_{KL}(d^\pi(s, a) \parallel d^E(s, a)) \quad (15)$$

$$= \mathbb{E}_{d^\pi} \left[\log \left(\frac{d^\pi(s, a)}{d^E(s, a)} \cdot \frac{d^{mix}(s, a)}{d^{mix}(s, a)} \right) \right] \quad (16)$$

$$= \mathbb{E}_{d^\pi} \left[\log \frac{d^{mix}(s, a)}{d^E(s, a)} \right] + \mathbb{E}_{d^\pi} \left[\log \frac{d^\pi(s, a)}{d^{mix}(s, a)} \right] \quad (17)$$

This decomposition does not require the coverage assumption because $d^{mix}(s, a)$ is always bigger than 0 whenever $d^E(s, a) > 0$. Therefore, minimizing the original KL divergence is equivalent to optimizing the following DICE objective:

$$\max_{d \geq 0} \mathbb{E}_d[r(s, a)] - D_{KL}(d(s, a) \parallel d^{mix}(s, a)) \quad (18)$$

s.t. Constraints (4)

where the reward function $r(s, a)$ is defined as $r(s, a) = -\log \left(\frac{d^{mix}(s, a)}{d^E(s, a)} \right)$. To estimate this reward function, we can estimate the density ratio $\frac{\beta d^E(s, a)}{d^{mix}(s, a)} = c^*(s, a)$, which corresponds to the optimal solution of the following discriminator training objective:

$$\max_c \beta \mathbb{E}_{d^E} [\log c(s, a)] + (1 - \beta) \mathbb{E}_{d^S} [\log(1 - c(s, a))] \quad (19)$$

Then the reward function can then be recovered as $r(s, a) = -\log \frac{\beta}{c^*(s, a)}$.

4.2. Dual Optimization via Gumbel Regression

As described in Section 3.1, optimizing the objective (18) with flow constraints can be done via solving its dual formulation:

$$\min_V (1 - \gamma) \mathbb{E}_{s \sim d_0} [V(s)] + \mathbb{E}_{d^{mix}} [f^*(\mathcal{T}V(s, a) - V(s))] \quad (20)$$

where $f^*(x) = \max\{0, (f')^{-1}(x)\}$ and $\mathcal{T}V(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s, a)} [V(s')]$.

For KL divergence, $f(x)$ can be $x \log x$, or $x \log x - x + 1$. (Schulman, 2020) empirically shows that the latter has lower variance than the former. Therefore, we use $f(x) = x \log x - x + 1$ which leads to $(f')^{-1}(x) = e^x$ and

$$f^*(x) = (\max\{0, e^x\})x - f(\max\{0, e^x\}) = e^x - 1$$

in our formulation. Furthermore, we propose to optimize this objective using a *Gumbel regression loss*: (Garg et al., 2023). We leverage the following result:

Proposition 4.1. *Optimizing the dual objective (20) under KL divergence is equivalent to minimizing the following Gumbel regression loss:*

$$\min_V \mathbb{E}_{(s, a) \sim d^{mix}} \left[e^{\mathcal{T}V(s, a) - V(s)} \right] - \mathbb{E}_{(s, a) \sim d^{mix}} [\mathcal{T}V(s, a) - V(s)] - 1 \quad (21)$$

Optimizing (21) offers several advantages. First, it avoids the need to estimate $(1 - \gamma) \mathbb{E}_{d_0} [V(s)]$, whereas both SMODICE and DemoDICE require sampling initial states from the dataset for this purpose. This requirement is not always feasible when initial-state data are limited or unavailable. A common workaround is to replace d_0 with d^S (Sikchi et al., 2024), which may work empirically but leads to biased estimation. Finally, the objective in (21) is strictly convex in V , guaranteeing a unique minimum in the V -space and thereby improving optimization stability and convergence during learning.

4.3. Policy Extraction

After learning the optimal value function $V^*(s)$, we extract the policy. Theoretically, the optimal policy can be extracted via the following weighted behavior cloning:

$$\max_{\pi} \mathbb{E}_{d^{mix}} [w(s, a) \log \pi(a|s)] \quad (22)$$

where $w(s, a) = \frac{d^{\pi^*}(s, a)}{d^{mix}(s, a)}$ is the optimal distribution correction defined as follows:

$$\begin{aligned} w(s, a) &= \max(0, (f')^{-1}(\mathcal{T}V(s, a) - V(s))) \\ &= \max(0, e^{\mathcal{T}V(s, a) - V(s)}) = e^{\mathcal{T}V(s, a) - V(s)} \end{aligned}$$

However, we find that directly using this weight does not result in good empirical performance. One possible explanation is that $V(s)$ can be overestimated, which causes the exponential weight to become negligibly small and thus suppress potentially useful data points. To mitigate this issue, we propose an alternative objective:

Proposition 4.2. *An optimal solution to the policy extraction (22) is also an optimal solution to the following learning objective:*

$$\max_{\pi} \mathbb{E}_{s,a \sim d^{mix}} \left[e^{\mathcal{T}V(s,a) - \tau V(s)} \log \pi(a|s) \right] \quad (23)$$

where $\tau \in [0, 1]$ is a scalar hyperparameter.

Setting $\tau < 1$ effectively reduces the influence of the value function on the current state, acting as a regularizer.

4.4. Practical Implementation

Computing the Bellman operator $\mathcal{T}V(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s,a)}[V(s')]$ requires the knowledge of transition dynamics $\mathcal{P}$ which is not available in offline setting. We follow previous works (Sikchi et al., 2024; Mao et al., 2024) using an empirical Bellman operator that only utilizes a single s' sample from datasets. Specifically, we optimize the following objective to learn the value function $V(s)$:

$$\min_V \mathcal{L}(V) = \mathbb{E}_{(s,a,s') \sim d^{mix}} \left[e^{r(s,a) + V(s') - V(s)} \right] - \mathbb{E}_{(s,a,s') \sim d^{mix}} [r(s,a) + V(s') - V(s)] - 1 \quad (24)$$

Note that, (21) and (24) are equivalent under deterministic transitions.

Optimizing (24) directly faces instability when trained via gradient descent. This instability can be understood as arising from conflicting gradient signals associated with $V(s)$ and $V(s')$, particularly when nearby states share similar learned feature representations due to feature co-adaptation (Kumar et al., 2022). To overcome this limitation, previous works (Xu et al., 2023; Sikchi et al., 2024) often use a semi-gradient update rule which avoids differentiating through the $V(s')$. Specifically, they incorporate another value function $Q(s,a)$ denoting a stop-gradient operator of $r(s,a) + V(s')$ and optimize Q via a MSE (mean squared error) loss. However, this increases the training cost due to adding another neural network. We instead adopt the orthogonal gradient update introduced in ODICE (Mao et al., 2024) for offline RL, which resolves gradient interference by decomposing the update into a projection of the gradient of and its orthogonal complement in a principled manner. We refer readers to ODICE for a detailed discussion. The full practical algorithm is summarized in Algorithm 1.

Algorithm 1 ReDICE

Input: Expert dataset $\mathcal{D}_E$, Suboptimal dataset $\mathcal{D}_S$, Hyperparameters γ, τ, β .

Initialize value, policy and discriminator networks $V_{\omega}, \pi_{\theta}, c_{\phi}$. Train discriminator c_{ϕ} via (19)

Compute rewards $r(s,a) = -\log \frac{\beta}{c_{\phi}(s,a)} \forall (s,a) \in \mathcal{D}_E \cup \mathcal{D}_S$

for step $t = 1$ to T **do**

 Sample mini-batch $B = \{(s,a,s')\} \sim \mathcal{D}_E \cup \mathcal{D}_S$.

Update Value Function: Train V_{ω} via Gumbel loss (Eq. 24) using orthogonal gradient update.

Policy Extraction: Update policy π_{θ}

$$\max_{\theta} \mathbb{E}_{d^{mix}} \left[e^{r(s,a) + \gamma V(s') - \tau V(s)} \log \pi_{\theta}(a|s) \right]$$

end for

5. Experiments

We empirically address following questions: **(Q1)** How well does ReDICE perform compared to prior state-of-the-art methods on standard offline IL benchmarks? **(Q2)** What is the performance of ReDICE on a limited expert coverage setting? **(Q3)** Can we apply ReDICE to a real-world scenario? **(Q4)** What are the effects of the proposed components, and key hyperparameters on ReDICE performance? The environment and implementation details are in the Appendix C.

Baselines and Experimental Setup We compare our method against several representative IL baselines. **BCE** trains a BC policy solely on expert demonstrations, while **BCU** extends BCE by training on both expert and suboptimal datasets. For DICE methods, we include **DemoDICE** (Kim et al., 2021) and **SMODICE-action**, which adapts SMODICE (Ma et al., 2022) to leverage action information. We also compare against **ReCOIL** (Sikchi et al., 2024), a state-of-the-art offline IL method that outperforms SMODICE; and **ILID** (Yue et al., 2024), a discriminator-weighted BC approach that has demonstrated strong performance in the challenging setting where suboptimal data have low expert coverage. Each method is trained for 1M gradient steps across 5 random seeds, with performance evaluated periodically using the *normalized score* = $100 * \frac{\text{method score} - \text{random score}}{\text{expert score} - \text{random score}}$ (Fu et al., 2020). For comparison, we report the average normalized score computed from the final 10 evaluations.

5.1. Offline IL

To address (Q1), we adopt the same offline IL benchmark used in ReCOIL (Sikchi et al., 2024), with datasets constructed from the D4RL framework (Fu et al., 2020). We evaluate all methods on eight MuJoCo (Todorov et al., 2012) environments, including four locomotion (*Hopper*, *HalfCheetah*, *Walker2d*, *Ant*) and four manipulation (*Pen*,

Table 1. Average normalized return over last 10 evaluations of ReDICE against baselines on the D4RL suboptimal datasets with 1 expert trajectory. The mean and std are obtained over 5 random seeds. Offline IL methods with avg. perf within the std-dev of the top performing approach is in **bold**.

Suboptimal Dataset	Env	BCE	BCU	DemoDICE	SMODICE-action	ILID	ReCOIL	ReDICE	Expert
random+ expert	hopper	17.87 $\pm$ 1.66	4.31 $\pm$ 0.17	99.85 $\pm$ 1.98	104.09 $\pm$ 1.82	99.57 $\pm$ 10.58	98.33 $\pm$ 12.97	109.80 $\pm$ 0.31	111.3
	halfcheetah	2.46 $\pm$ 0.45	4.06 $\pm$ 0.93	81.97 $\pm$ 3.84	80.92 $\pm$ 1.94	3.50 $\pm$ 1.07	78.78 $\pm$ 6.71	91.82 $\pm$ 0.44	88.83
	walker2d	11.42 $\pm$ 5.22	1.40 $\pm$ 0.13	107.89 $\pm$ 0.24	107.46 $\pm$ 0.22	107.81 $\pm$ 0.23	105.72 $\pm$ 3.55	107.98 $\pm$ 0.12	106.92
	ant	24.74 $\pm$ 1.47	37.82 $\pm$ 5.05	126.80 $\pm$ 0.71	126.48 $\pm$ 1.78	90.72 $\pm$ 4.52	116.43 $\pm$ 3.69	126.32 $\pm$ 0.81	130.75
random+ few-expert	hopper	17.87 $\pm$ 1.66	4.11 $\pm$ 0.13	78.15 $\pm$ 8.40	58.72 $\pm$ 7.09	99.79 $\pm$ 14.15	86.65 $\pm$ 20.85	93.63 $\pm$ 1.69	111.33
	halfcheetah	2.46 $\pm$ 0.45	2.24 $\pm$ 0.02	16.63 $\pm$ 6.75	4.16 $\pm$ 2.83	2.22 $\pm$ 0.02	5.25 $\pm$ 0.49	79.31 $\pm$ 4.59	88.83
	walker2d	11.42 $\pm$ 5.22	1.07 $\pm$ 0.24	99.54 $\pm$ 8.26	3.48 $\pm$ 2.57	106.87 $\pm$ 1.42	12.59 $\pm$ 11.04	108.26 $\pm$ 0.03	106.92
	ant	24.74 $\pm$ 1.47	37.73 $\pm$ 6.58	89.11 $\pm$ 6.83	13.26 $\pm$ 5.01	82.34 $\pm$ 7.25	56.68 $\pm$ 15.22	108.15 $\pm$ 3.59	130.75
medium+ expert	hopper	17.87 $\pm$ 1.66	54.69 $\pm$ 0.45	81.23 $\pm$ 12.90	57.83 $\pm$ 3.52	34.10 $\pm$ 13.25	85.25 $\pm$ 20.86	111.01 $\pm$ 0.38	111.33
	halfcheetah	2.46 $\pm$ 0.45	42.61 $\pm$ 0.20	75.59 $\pm$ 3.56	66.52 $\pm$ 2.32	43.26 $\pm$ 1.65	78.88 $\pm$ 3.56	91.80 $\pm$ 0.42	88.83
	walker2d	11.42 $\pm$ 5.22	73.43 $\pm$ 1.66	106.65 $\pm$ 0.64	39.71 $\pm$ 10.78	75.56 $\pm$ 51.32	108.27 $\pm$ 1.46	108.40 $\pm$ 0.50	106.92
	ant	24.74 $\pm$ 1.47	97.37 $\pm$ 4.08	114.56 $\pm$ 4.50	109.73 $\pm$ 1.88	73.30 $\pm$ 16.61	119.21 $\pm$ 3.71	123.24 $\pm$ 1.48	130.75
medium+ few-expert	hopper	17.87 $\pm$ 1.66	52.51 $\pm$ 0.52	61.65 $\pm$ 1.97	53.33 $\pm$ 0.80	20.98 $\pm$ 6.43	38.88 $\pm$ 25.99	109.50 $\pm$ 1.23	111.33
	halfcheetah	2.46 $\pm$ 0.45	42.52 $\pm$ 0.28	47.41 $\pm$ 2.99	41.47 $\pm$ 1.99	42.67 $\pm$ 1.21	41.90 $\pm$ 3.37	81.57 $\pm$ 2.94	88.83
	walker2d	11.42 $\pm$ 5.22	72.26 $\pm$ 1.48	87.02 $\pm$ 6.02	13.78 $\pm$ 14.21	82.67 $\pm$ 9.96	71.44 $\pm$ 15.45	108.70 $\pm$ 0.44	106.92
	ant	24.74 $\pm$ 1.47	88.81 $\pm$ 0.54	97.71 $\pm$ 0.85	89.22 $\pm$ 2.44	71.08 $\pm$ 10.07	104.03 $\pm$ 4.74	111.73 $\pm$ 2.37	130.75
cloned+expert	pen	8.84 $\pm$ 1.64	25.75 $\pm$ 3.23	36.91 $\pm$ 5.75	33.97 $\pm$ 19.99	7.07 $\pm$ 5.70	45.59 $\pm$ 9.17	88.84 $\pm$ 8.84	106.42
	door	0.02 $\pm$ 0.07	0.13 $\pm$ 0.20	0.07 $\pm$ 0.07	3.57 $\pm$ 3.38	-0.03 $\pm$ 0.04	16.18 $\pm$ 17.29	92.23 $\pm$ 2.45	103.94
	hammer	0.32 $\pm$ 0.14	4.29 $\pm$ 3.72	0.91 $\pm$ 0.74	3.98 $\pm$ 5.45	0.19 $\pm$ 0.11	2.60 $\pm$ 1.35	104.86 $\pm$ 10.07	125.71
human+expert	pen	8.84 $\pm$ 1.64	27.07 $\pm$ 3.91	39.70 $\pm$ 8.36	46.08 $\pm$ 3.85	11.65 $\pm$ 7.85	85.04 $\pm$ 5.56	82.62 $\pm$ 3.58	106.42
	door	0.02 $\pm$ 0.07	8.77 $\pm$ 6.73	18.37 $\pm$ 7.47	26.16 $\pm$ 11.76	-0.05 $\pm$ 0.02	84.02 $\pm$ 11.42	94.31 $\pm$ 1.83	103.94
	hammer	0.32 $\pm$ 0.14	30.37 $\pm$ 14.03	27.38 $\pm$ 5.80	49.13 $\pm$ 13.31	1.56 $\pm$ 2.53	86.45 $\pm$ 14.59	107.97 $\pm$ 4.85	125.71
partial+expert	kitchen	4.58 $\pm$ 6.23	36.25 $\pm$ 2.50	46.67 $\pm$ 2.55	38.33 $\pm$ 7.64	45.50 $\pm$ 6.38	48.80 $\pm$ 4.00	54.00 $\pm$ 1.32	75.00
mixed+expert	kitchen	4.58 $\pm$ 6.23	44.50 $\pm$ 1.39	45.67 $\pm$ 2.75	46.67 $\pm$ 5.20	2.58 $\pm$ 4.26	49.35 $\pm$ 1.66	48.25 $\pm$ 0.35	75.00
Average		10.56	33.09	66.14	50.75	46.04	67.76	97.68	107.23

Door, Hammer, Kitchen). For each task, the expert dataset consists of a single trajectory. In the locomotion domains, suboptimal datasets are formed by combining D4RL random or medium data with either 200 expert trajectories (*expert*) or 30 expert trajectories (*few-expert*). For manipulation tasks, non-expert datasets from D4RL (mixed and partial for Kitchen; human and cloned for the remaining tasks) are combined with up to 30 expert trajectories. This construction yields 24 diverse tasks for benchmarking. Manipulation and ‘few-expert’ tasks pose greater challenges due to their higher-dimensional state spaces and low expert coverage. Additional details on environments and dataset construction are provided in the Appendix C.1.

Table 1 summarizes the average normalized score of ReDICE and prior offline IL baselines. Overall, ReDICE achieves the best aggregate performance (97.68 average), outperforming the strongest baseline on average by about 44% (vs. ReCOIL: 67.76), and reaching 91.1% of the expert average (107.23). ReDICE is the top-performing method on all tasks. The gains are especially considerable in the few-expert setting, where several baselines collapse (e.g., on halfcheetah under random+few-expert), while ReDICE retains strong near-expert performance. On the more challenging manipulation tasks like pen, door and hammer, ReDICE substantially outperforms prior methods.

5.2. Offline IL with Limited Expert Coverage

To address (Q2), we consider an even more challenging setting with *very low expert coverage*, similar to the setup studied in ILID (Yue et al., 2024). The expert dataset still contains only a single trajectory, but unlike the benchmark in Section 5.1, the suboptimal dataset is *not* mixed with any expert rollouts and is instead collected purely from low-quality behaviors (e.g., D4RL random/medium or cloned/human). This setting stresses an algorithm’s ability to correctly propagate sparse expert information.

Table 2 reports normalized scores for this setting. We additionally include an **OfflineRL** column as an approximate upper bound, constructed from the best score among multiple offline RL methods reported in ReBRAC (Tarasov et al., 2023) when trained on the same suboptimal datasets with access to ground-truth rewards. For Ant tasks that are not reported in ReBRAC, we run ReBRAC ourselves using a sweep of configurations adapted from the other locomotion tasks and report the best score. Overall, ReDICE is close to methods with access to true rewards, while consistently outperforming prior offline IL baselines. The gains are particularly large on tasks where other DICE-based methods struggle (e.g., HalfCheetah and manipulation tasks). These results clearly demonstrate that ReDICE does not rely on the coverage assumption and can therefore learn from diverse behavior in the suboptimal dataset.

Table 2. Average normalized score over last 10 evaluations of ReDICE against baselines on the *limited expert coverage* setting, with 1 expert trajectory and *no expert rollouts mixed into the suboptimal dataset*. The mean and std are obtained over 5 random seeds. Offline IL methods with avg. perf within the std-dev of the top performing approach is in **bold**.

Suboptimal Dataset	Env	BCE	BCU	DemoDICE	SMODICE-action	ILID	ReCOIL	ReDICE	OfflineRL
random	hopper	17.87 $\pm$ 1.66	2.47 $\pm$ 0.33	4.94 $\pm$ 0.42	17.77 $\pm$ 7.53	23.49 $\pm$ 15.37	19.97 $\pm$ 12.62	25.36 $\pm$ 7.23	19.6
	halfcheetah	2.46 $\pm$ 0.45	2.24 $\pm$ 0.00	2.21 $\pm$ 0.02	2.15 $\pm$ 0.06	2.25 $\pm$ 0.01	2.28 $\pm$ 0.23	16.79 $\pm$ 1.13	31.1
	walker2d	11.42 $\pm$ 5.22	0.70 $\pm$ 0.23	10.73 $\pm$ 2.88	-0.25 $\pm$ 0.19	17.04 $\pm$ 14.10	6.33 $\pm$ 0.64	15.59 $\pm$ 1.44	18.7
	ant	24.74 $\pm$ 1.47	30.74 $\pm$ 0.53	27.48 $\pm$ 1.15	27.05 $\pm$ 3.27	30.90 $\pm$ 3.18	31.50 $\pm$ 0.03	36.65 $\pm$ 3.35	49.84
medium	hopper	17.87 $\pm$ 1.66	52.45 $\pm$ 1.42	70.73 $\pm$ 3.68	53.99 $\pm$ 0.79	23.06 $\pm$ 8.00	36.10 $\pm$ 8.89	82.13 $\pm$ 2.54	102
	halfcheetah	2.46 $\pm$ 0.45	42.44 $\pm$ 0.22	41.20 $\pm$ 0.58	40.59 $\pm$ 0.51	42.76 $\pm$ 0.54	35.06 $\pm$ 1.04	49.58 $\pm$ 0.28	66.4
	walker2d	11.42 $\pm$ 5.22	64.86 $\pm$ 4.50	70.73 $\pm$ 3.68	3.94 $\pm$ 5.29	67.69 $\pm$ 2.42	61.16 $\pm$ 2.55	66.22 $\pm$ 2.97	92.7
	ant	24.74 $\pm$ 1.47	89.80 $\pm$ 2.35	70.73 $\pm$ 3.68	-32.07 $\pm$ 25.45	84.38 $\pm$ 4.14	92.17 $\pm$ 2.99	99.05 $\pm$ 1.04	133.31
cloned	pen	8.84 $\pm$ 1.64	22.91 $\pm$ 3.99	29.76 $\pm$ 0.98	14.16 $\pm$ 21.06	2.76 $\pm$ 8.70	30.67 $\pm$ 2.38	31.65 $\pm$ 1.58	91.8
	door	0.02 $\pm$ 0.07	0.01 $\pm$ 0.00	0.17 $\pm$ 0.04	0.94 $\pm$ 1.54	-0.11 $\pm$ 0.10	0.12 $\pm$ 0.20	5.04 $\pm$ 1.49	1.1
	hammer	0.32 $\pm$ 0.14	0.51 $\pm$ 0.23	0.41 $\pm$ 0.08	0.65 $\pm$ 0.45	0.24 $\pm$ 0.04	0.53 $\pm$ 0.35	19.89 $\pm$ 11.67	6.7
human	pen	8.84 $\pm$ 1.64	34.43 $\pm$ 2.63	50.51 $\pm$ 3.43	13.65 $\pm$ 7.11	7.36 $\pm$ 8.77	70.83 $\pm$ 9.13	77.59 $\pm$ 2.25	103.5
	door	0.02 $\pm$ 0.07	0.44 $\pm$ 0.43	1.12 $\pm$ 0.73	0.41 $\pm$ 0.35	-0.28 $\pm$ 0.06	2.87 $\pm$ 0.92	7.56 $\pm$ 2.30	9.9
	hammer	0.32 $\pm$ 0.14	0.81 $\pm$ 0.37	1.21 $\pm$ 0.78	0.69 $\pm$ 0.09	0.57 $\pm$ 1.08	4.68 $\pm$ 0.87	5.22 $\pm$ 1.02	4.4
Average		9.38	24.63	27.28	10.26	21.58	28.16	38.45	52.22

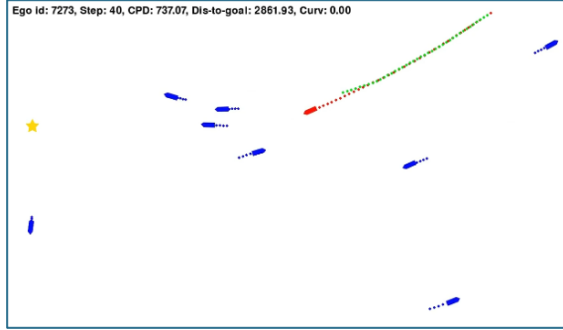


Figure 1. Illustration of an episode from the maritime simulation environment. Vessels shown in blue (log-play agents) move along their historical trajectories, the red vessel is controlled by an IL policy with its destination marked by *, and green dots indicate the ego vessel’s historical trajectory in the dataset.

5.3. Application to Maritime Navigation

We evaluate ReDICE on a real-world maritime navigation task where the objective is to imitate how professional pilots maneuver large ships (primarily tankers and cargo vessels) in dense traffic. Beyond serving as a realistic benchmark, learned vessel-behavior models can support safety-critical operations: (i) they can be plugged into a VTIS (Vessel Traffic Information System) pipeline to forecast short-term motions of ships in congestion hotspots such as the Singapore Strait, and (ii) they enable counterfactual stress tests (e.g., simulating a temporary increase in arrivals and assigning learned policies to the injected traffic). Existing RL-based approaches to maritime traffic management are commonly trained online, which demands extensive simulator interaction and high-fidelity vessel dynamics (Singh et al., 2020). Our setting instead focuses on learning directly from offline navigation logs.

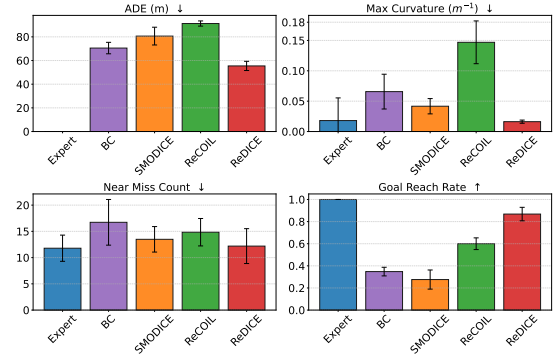


Figure 2. Maritime navigation results in ShipNaviSim. Mean and standard deviation are over 5 random seeds.

We use roughly two years of vessel trajectories in a high-traffic area recorded by the Automatic Identification System (AIS). We instantiate the learning environment using **ShipNaviSim** (Pham et al., 2025), where a single ego vessel is controlled by the learned IL policy and other nearby vessels are replayed from logs (log-play agents), yielding realistic multi-ship encounters (Figure 1). The goal of ego vessel is reaching the destination while avoiding collision with other vessels, and its observation includes a short history of its own states along with the states of nearby vessels. This makes the setting particularly challenging due to the high-dimensional observation space (200 dimensions in our configuration). Since AIS logs provide state sequences but not control inputs, we recover an action representation from consecutive states using the same kinematic inverse model as ShipNaviSim, which allows us to train offline IL methods. We treat AIS trajectories as expert demonstrations, and construct suboptimal data by adding random noise into expert dataset. A comprehensive description of the environment and dataset generation is presented in the Appendix C.2.

Figure 2 shows the comparison results for this setting between ReDICE and other methods. Following the evaluation protocol in ShipNaviSim, we report: **ADE** (Average Displacement Error measuring the difference between trajectories generated by expert and IL policy; lower is better); **goal reach rate** (higher is better), **near-miss count** (lower is better; counts close-quarter events during an episode), and kinematic realism via **max curvature** (smaller values indicate less abrupt turning and more physically plausible maneuvers). All results are averaged over five random seeds, with error bars indicating standard deviation. Compared to the expert, all baseline methods including BC, SMODICE, and ReCOIL exhibit substantially lower goal reach rates ($\approx 60\%$), higher curvature, and increased near-miss counts, indicating difficulties in producing smooth, safe, and reliable vessel behaviors. These shortcomings highlight the challenges of learning from offline data in high-dimensional maritime environment. In contrast, ReDICE consistently improves across all metrics, producing smoother trajectories, fewer near-miss events, and a much higher goal reach rate ($\approx 85\%$), substantially narrowing the gap to the expert. This shows that ReDICE is effective not only on standard benchmarks but also for real-world applications such as maritime navigation

5.4. Ablation Study

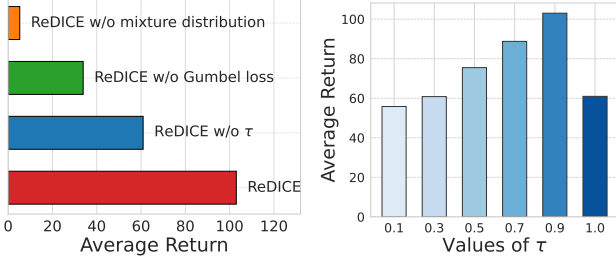


Figure 3. Ablation results on different components (Left), and τ values (Right)

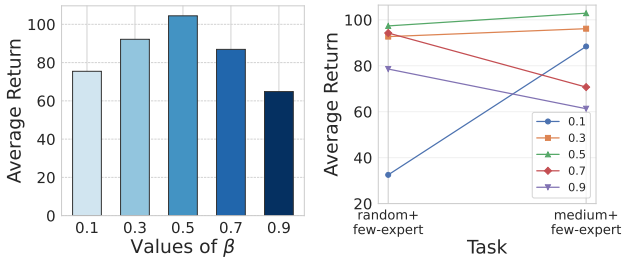


Figure 4. Ablation results on different β

We address (Q4) through an ablation study of ReDICE’s key components and hyperparameters τ and β . Results are shown in Figures 3 and 4, with additional details in Appendix D.

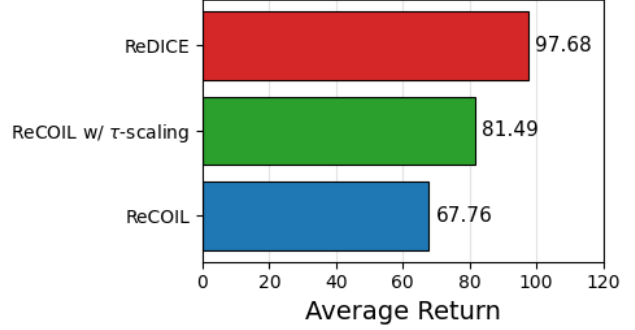


Figure 5. Effect of τ -scaling on ReCOIL compared with ReDICE. Results are averaged over 5 random seeds across all D4RL suboptimal tasks.

Analysis of proposed components: Figure 3 (left) compares the average return across tasks achieved by ReDICE and several ablated variants, including removing the mixture distribution, the Gumbel-based loss, and the temperature parameter τ . We observe a significant performance degradation when any of these components is removed, indicating that each plays a critical role in the overall effectiveness of ReDICE. In particular, removing the mixture distribution leads to the largest drop (about 20 times) in performance, highlighting the importance of balancing expert and suboptimal data for stable learning.

Analysis of τ : According to Figure 3 (right), increasing τ value initially leads to improved performance by allowing the policy to place greater emphasis on high-advantage actions. However, the performance peaks at $\tau = 0.9$, after which further increases yield diminishing or negative returns. This drop is likely due to overly aggressive advantage weighting, which makes the policy extraction sensitive to errors in the value function. This indicates that $\tau = 0.9$ strikes a favorable balance between stable optimization and effective policy improvement, which is the reason we choose this τ value for most tasks. In practice, we do not tune τ over a large range. Following the common practice of tuning policy temperatures in offline methods, we use a small candidate set $\tau \in \{0.9, 1.0\}$. We use $\tau = 0.9$ for most tasks and $\tau = 1.0$ only for a few cases such as Kitchen and human+expert manipulation tasks. Thus, the proposed policy extraction introduces only minimal additional tuning cost.

Analysis of τ -scaling in ReCOIL: To further disentangle whether the performance gain comes from the proposed objective or from τ -scaled policy extraction alone, we additionally apply our τ -scaled policy extraction to ReCOIL. Results in Figure 5 show that τ -scaling consistently improves ReCOIL, increasing its average return from 67.76 to 81.49, corresponding to an approximately 20% relative

gain. This demonstrates that our policy extraction mechanism is not specific to ReDICE and can also improve other DICE-based offline IL objectives. However, ReCOIL with τ -scaling still underperforms ReDICE on most tasks, with an average return gap of approximately 20% compared to ReDICE (81.49 vs. 97.68). This indicates that the improvements of ReDICE mainly come from the core objective design rather than from τ -scaling alone.

Analysis of β : In Figure 4, we study the effect of varying the mixture coefficient β . We use $\beta = 0.5$ as the default value in all main experiments, following common practice in prior mixture-based IL works such as ReCOIL and related methods. Importantly, this default is not selected based on the ablation results; instead, the ablation is intended to evaluate the robustness of this fixed design choice.

Results show that $\beta = 0.5$ achieves the best average return across tasks, suggesting that equally balancing expert and suboptimal data provides a robust default. Extreme values such as $\beta = 0.1$ or $\beta = 0.9$ can bias learning too heavily toward either suboptimal or expert data, leading to lower average performance. The right panel of Figure 4 further indicates that the effect of β depends on the quality and coverage of the supplementary data. For larger β values (e.g., 0.7 or 0.9), performance under `random+few-expert`, which has lower expert coverage, is higher than under `medium+few-expert`, whereas the opposite trend holds for smaller β values (e.g., 0.1 or 0.3). This suggests that larger β values can be more beneficial when expert coverage is limited, while smaller β values better exploit informative suboptimal data when coverage is higher.

Overall, $\beta = 0.5$ provides a strong default across settings without requiring prior knowledge of supplementary data quality. Nevertheless, as in many offline IL/RL methods, a fixed hyperparameter may be suboptimal in some outlier cases. When additional validation is available and higher performance is required, we recommend optionally tuning β over a small candidate set $\{0.3, 0.5, 0.7\}$.

6. Conclusion

We revisited distribution correction estimation for offline imitation learning with supplementary suboptimal data and introduced ReDICE, a stable DICE-based approach derived from a KL-based objective under a mixture distribution. By using the mixture distribution as a reference measure, ReDICE relaxes the coverage assumption required by prior DICE-based offline IL methods while preserving the original expert-imitation objective. We further derive an equivalent dual formulation that reduces to a Gumbel regression objective, improving numerical stability, and propose a τ -scaled policy extraction mechanism that improves performance in practice. Empirically, ReDICE achieves state-of-

the-art performance across standard offline IL benchmarks, remains robust under limited expert coverage, and applies effectively to a real-world maritime navigation setting.

Limitations and Future Work. ReDICE has several limitations that open directions for future work. First, as with prior DICE-based methods, the practical objective relies on an empirical Bellman operator, which is exact under deterministic transitions but may introduce approximation error in highly stochastic environments. Second, ReDICE assumes that expert and supplementary data are collected from the same MDP; extending the method to cross-domain settings with different dynamics may require additional representation alignment or domain adaptation. Third, although we use $\beta = 0.5$ as a robust default following common practice, some outlier tasks may benefit from limited tuning of the mixture coefficient. Finally, ReDICE uses a discriminator to estimate density ratios, and more robust density-ratio estimation methods may further improve performance in high-dimensional or extremely low-data regimes. Future work will study these directions, including stochastic environments, adaptive mixture selection, and larger-scale vision-based offline IL benchmarks such as V-D4RL (Lu et al., 2023).

Acknowledgements

This work is supported by the Lee Kong Chian Fellowship awarded to Tien Mai.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning, specifically offline imitation learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Black, K., Brown, N., Darpanian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M. R., Finn, C., Fusai, N., Galliker, M. Y., Ghosh, D., Groom, L., Hausman, K., brian ichter, Jakubczak, S., Jones, T., Ke, L., LeBlanc, D., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Tanner, J., Vuong, Q., Walke, H., Walling, A., Wang, H., Yu, L., and Zhilinsky, U. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *9th Annual Conference on Robot Learning*, 2025.
- Cheng, J., Chen, Y., Mei, X., Yang, B., Li, B., and Liu, M. Rethinking imitation-based planners for autonomous

- driving. In *International Conference on Robotics and Automation (ICRA)*, 2024.
- Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- Garg, D., Chakraborty, S., Cundy, C., Song, J., and Ermon, S. Iq-learn: Inverse soft-q learning for imitation. *Advances in Neural Information Processing Systems*, 2021.
- Garg, D., Hejna, J., Geist, M., and Ermon, S. Extreme q-learning: Maxent RL without entropy. In *International Conference on Learning Representations*, 2023.
- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial nets. *Advances in neural information processing systems*, 2014.
- Gulino, C., Fu, J., Luo, W., Tucker, G., Bronstein, E., Lu, Y., Harb, J., Pan, X., Wang, Y., Chen, X., et al. Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research. *Advances in Neural Information Processing Systems*, 2023.
- Kim, G.-H., Seo, S., Lee, J., Jeon, W., Hwang, H., Yang, H., and Kim, K.-E. Demodice: Offline imitation learning with supplementary imperfect demonstrations. In *International Conference on Learning Representations*, 2021.
- Kostrikov, I., Nachum, O., and Tompson, J. Imitation learning via off-policy distribution matching. In *International Conference on Learning Representations*, 2020.
- Kumar, A., Zhou, A., Tucker, G., and Levine, S. Conservative q-learning for offline reinforcement learning. *Advances in neural information processing systems*, 2020.
- Kumar, A., Agarwal, R., Ma, T., Courville, A., Tucker, G., and Levine, S. DR3: Value-based deep reinforcement learning requires explicit regularization. In *International Conference on Learning Representations*, 2022.
- Lee, J., Jeon, W., Lee, B., Pineau, J., and Kim, K.-E. Optidice: Offline policy optimization via stationary distribution correction estimation. In *International Conference on Machine Learning*, 2021.
- Lee, J., Paduraru, C., Mankowitz, D. J., Heess, N., Precup, D., Kim, K.-E., and Guez, A. COptiDICE: Offline constrained reinforcement learning via stationary distribution correction estimation. In *International Conference on Learning Representations*, 2022.
- Lee, J., Sun, M., and Abbeel, P. SEMDICE: Off-policy state entropy maximization via stationary distribution correction estimation. In *International Conference on Learning Representations*, 2025.
- Li, J., Hu, X., Xu, H., Liu, J., Zhan, X., Jia, Q.-S., and Zhang, Y.-Q. Mind the gap: Offline policy optimization for imperfect rewards. In *International Conference on Learning Representations*, 2023a.
- Li, Z., Xu, T., Qin, Z., Yu, Y., and Luo, Z.-Q. Imitation learning from imperfection: Theoretical justifications and algorithms. *Advances in Neural Information Processing Systems*, 2023b.
- Lu, C., Ball, P. J., Rudner, T. G. J., Parker-Holder, J., Osborne, M. A., and Teh, Y. W. Challenges and opportunities in offline reinforcement learning from visual observations. *Transactions on Machine Learning Research*, 2023.
- Ma, Y., Shen, A., Jayaraman, D., and Bastani, O. Versatile offline imitation from observations and examples via regularized state-occupancy matching. In *International Conference on Machine Learning*, 2022.
- Manne, A. S. Linear programming and sequential decisions. *Management Science*, 1960.
- Mao, L., Xu, H., Zhang, W., and Zhan, X. ODICE: Revealing the mystery of distribution correction estimation via orthogonal-gradient update. In *International Conference on Learning Representations*, 2024.
- Nachum, O. and Dai, B. Reinforcement learning via fenchel-rockafellar duality. *arXiv preprint arXiv:2001.01866*, 2020.
- Nachum, O., Chow, Y., Dai, B., and Li, L. Dualdice: Behavior-agnostic estimation of discounted stationary distribution corrections. *Advances in neural information processing systems*, 2019.
- Pham, Q. A., Brahmanage, J. C., and Kumar, A. Shipnavisim: Data-driven simulation for real-world maritime navigation. In *International Conference on Autonomous Agents and Multiagent Systems*, 2025.
- Pomerleau, D. A. Alvin: An autonomous land vehicle in a neural network. *Advances in neural information processing systems*, 1988.
- Ross, S., Gordon, G., and Bagnell, D. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, 2011.
- Schulman, J. Approximating kl divergence. Blog post, 2020. URL <http://joschu.net/blog/kl-approx.html>. Accessed: 2025-01-06.

- Sikchi, H., Zheng, Q., Zhang, A., and Niekum, S. Dual RL: Unification and new methods for reinforcement and imitation learning. In *International Conference on Learning Representations*, 2024.
- Singh, A. J., Kumar, A., and Lau, H. C. Hierarchical multiagent reinforcement learning for maritime traffic management. In *International Conference on Autonomous Agents and MultiAgent Systems*, 2020.
- Tarasov, D., Kurenkov, V., Nikulin, A., and Kolesnikov, S. Revisiting the minimalist approach to offline reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.
- Todorov, E., Erez, T., and Tassa, Y. Mujoco: A physics engine for model-based control. In *International conference on intelligent robots and systems*, 2012.
- Xu, H., Zhan, X., Yin, H., and Qin, H. Discriminator-weighted offline imitation learning from suboptimal demonstrations. In *International Conference on Machine Learning*, 2022.
- Xu, H., Jiang, L., Li, J., Yang, Z., Wang, Z., Chan, V. W. K., and Zhan, X. Offline RL with no OOD actions: In-sample learning via implicit value regularization. In *International Conference on Learning Representations*, 2023.
- Yang, M., Nachum, O., Dai, B., Li, L., and Schuurmans, D. Off-policy evaluation via the regularized lagrangian. *Advances in Neural Information Processing Systems*, 2020.
- Yu, L., Yu, T., Song, J., Neiswanger, W., and Ermon, S. Offline imitation learning with suboptimal demonstrations via relaxed distribution matching. In *Proceedings of the AAAI conference on artificial intelligence*, 2023.
- Yue, S., Liu, J., Hua, X., Ren, J., Lin, S., Zhang, J., and Zhang, Y. How to leverage diverse demonstrations in offline imitation learning. In *International Conference on Machine Learning*, 2024.
- Zare, M., Kebria, P. M., Khosravi, A., and Nahavandi, S. A survey of imitation learning: Algorithms, recent developments, and challenges. *IEEE Transactions on Cybernetics*, 2024.
- Zhang, R., Dai, B., Li, L., and Schuurmans, D. Gendice: Generalized offline estimation of stationary values. In *International Conference on Learning Representations*, 2020.

A. Missing Proofs and Derivations

A.1. Complete derivation of transforming objective function (3) to (5)

We recall that the primal DICE formulation is as follows:

$$\begin{aligned} & \max_{d \geq 0} \mathbb{E}_{s,a \sim d} [r(s, a)] - \alpha D_f(d(s, a) \| d^D(s, a)) \\ \text{s.t. } & \sum_a d(s, a) = (1 - \gamma) d_0(s) + \gamma \mathcal{T}_* d(s), \quad \forall s \end{aligned}$$

where $\mathcal{T}_* d(s) = \sum_{\bar{s}, \bar{a}} d(\bar{s}, \bar{a}) \mathcal{P}(s | \bar{s}, \bar{a})$. We apply Lagrangian duality to convert the primal formulation into an unconstrained problem with dual variables V :

$$\begin{aligned} & \min_{V(s)} \max_{d \geq 0} \mathbb{E}_{(s,a) \sim d} [r(s, a) - \alpha D_f(d(s, a) \| d^D(s, a))] + \sum_s V(s) \left((1 - \gamma) d_0(s) + \gamma \mathcal{T}_* d(s) - \sum_a d(s, a) \right) \\ & = \min_{V(s)} \max_{d \geq 0} (1 - \gamma) \sum_s d_0(s) V(s) + \mathbb{E}_{(s,a) \sim d} [r(s, a)] + \sum_{s,a} d(s, a) \sum_{s'} V(s') \mathcal{P}(s' | s, a) \\ & \quad - \sum_{s,a} d(s, a) V(s) - \alpha D_f(d(s, a) \| d^D(s, a)) \end{aligned} \quad (25)$$

$$\begin{aligned} & = \min_{V(s)} \max_{w \geq 0} (1 - \gamma) \mathbb{E}_{d_0(s)} [V(s)] + \mathbb{E}_{s,a \sim d^D} \left[w(s, a) \left(r(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) V(s') - V(s) \right) \right] \\ & \quad - \mathbb{E}_{s,a \sim d^D} [\alpha f(w(s, a))]. \end{aligned} \quad (26)$$

where $w(s, a) = \frac{d(s, a)}{d^D(s, a)}$ is often known as a distribution correction or density ratio. We first focus on solving the inner maximization of (26). Let's denote $\delta(s, a, V) = r(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) V(s') - V(s)$. We can relax the constraint $w \geq 0$ via duality:

$$\min_{\lambda \geq 0} \max_{w(s, a)} \mathbb{E}_{s,a \sim d^D} [w(s, a) \delta(s, a, V)] - \mathbb{E}_{s,a \sim d^D} [\alpha f(w(s, a))] + \sum_{s,a} \lambda(s, a) w(s, a). \quad (27)$$

in which $\lambda(s, a)$ are the Lagrange multipliers. Since strong duality holds, we can solve the following KKT conditions:

$$w^*(s, a) \geq 0 \quad \forall s, a \quad (\text{primal feasibility}) \quad (28)$$

$$\lambda^*(s, a) \geq 0 \quad \forall s, a \quad (\text{dual feasibility}) \quad (29)$$

$$d^D(s, a) [-\alpha f'(w^*(s, a)) + \delta(s, a, V) + \lambda^*(s, a)] = 0 \iff \alpha f'(w(s, a)) = \delta(s, a, V) + \lambda^*(s, a) \quad \forall s, a \quad (\text{stationary}) \quad (30)$$

$$\lambda^*(s, a) w^*(s, a) = 0 \quad \forall s, a \quad (\text{complementary slackness}) \quad (31)$$

Applying these conditions, we can derive a closed-form solution of the distribution correction $w^*(s, a) = \max\{0, (f')^{-1}(\frac{\delta(s, a, V)}{\alpha})\}$. This allows us to omit w in (26) to obtain an optimization problem depending only on V :

$$\min_V (1 - \gamma) \mathbb{E}_{s \sim d_0} [V(s)] + \alpha \mathbb{E}_{s,a \sim d^D} [f^*\left(\frac{\delta(s, a, V)}{\alpha}\right)] \quad (32)$$

where $f^*(x) = \max\{0, (f')^{-1}(x)\} \cdot x - f(\max\{0, (f')^{-1}(x)\})$ is a variant of convex conjugate.

Let's denote $\mathcal{TV}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot | s, a)} [V(s')]$ as a Bellman operator on V . $\delta(s, a, V)$ is equal to $\mathcal{TV}(s, a) - V(s)$. The objective (32) becomes:

$$\min_V (1 - \gamma) \mathbb{E}_{s \sim d_0} [V(s)] + \alpha \mathbb{E}_{s,a \sim d^D} [f^*\left(\frac{\mathcal{TV}(s, a) - V(s)}{\alpha}\right)] \quad (33)$$

A.2. Proof of Proposition 4.1

Corollary A.1.1 (Garg et al., 2021) For any valid occupancy measure μ over state-action pairs and any value function V , it holds that

$$\mathbb{E}_{(s,a) \sim \mu} [V(s) - \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s,a)} [V(s')]] = (1 - \gamma) \mathbb{E}_{s \sim p_0} [V(s)]. \quad (34)$$

We use this Corollary to prove the proposition as follows:

Proof. We can derive the following result from (34):

$$(1 - \gamma) \mathbb{E}_{s \sim p_0} [V(s)] = -\mathbb{E}_{(s,a) \sim \mu} [\mathcal{T}V(s, a) - V(s) - r(s, a)] \quad (35)$$

where $\mathcal{T}V(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s,a)} [V(s')]$.

Applying this result into the (20) with $f^*(x) = e^x - 1$ we have:

$$\begin{aligned} & (1 - \gamma) \mathbb{E}_{s \sim d_0} [V(s)] + \mathbb{E}_{s,a \sim d^{mix}} [f^*(\mathcal{T}V(s, a) - V(s))] \\ &= (1 - \gamma) [\beta \mathbb{E}_{s \sim d_0} [V(s)] + (1 - \beta) \mathbb{E}_{s \sim d_0} [V(s)]] + \mathbb{E}_{s,a \sim d^{mix}} [e^{\mathcal{T}V(s,a) - V(s)} - 1] \\ &= -\beta \mathbb{E}_{(s,a) \sim d^E} [\mathcal{T}V(s, a) - V(s) - r(s, a)] - (1 - \beta) \mathbb{E}_{(s,a) \sim d^S} [\mathcal{T}V(s, a) - V(s) - r(s, a)] \\ &+ \mathbb{E}_{d^{mix}} [e^{\mathcal{T}V(s,a) - V(s)}] - 1 \\ &= \mathbb{E}_{d^{mix}} [e^{\mathcal{T}V(s,a) - V(s)}] - \mathbb{E}_{(s,a) \sim d^{mix}} [\mathcal{T}V(s, a) - V(s) - r(s, a)] - 1 \\ &= \mathbb{E}_{d^{mix}} [e^{\mathcal{T}V(s,a) - V(s)}] - \mathbb{E}_{(s,a) \sim d^{mix}} [\mathcal{T}V(s, a) - V(s)] - 1 + \mathbb{E}_{(s,a) \sim d^{mix}} [r(s, a)] \end{aligned}$$

Because $r(s, a)$ is a fixed term inferred from a pretrained discriminator, and d^{mix} is a mixture of occupancy measures of fixed datasets, optimizing (20) is equivalent to the following Gumbel regression loss (Garg et al., 2023):

$$\min_V \mathbb{E}_{d^{mix}} [e^{\mathcal{T}V(s,a) - V(s)}] - \mathbb{E}_{(s,a) \sim d^{mix}} [\mathcal{T}V(s, a) - V(s)] - 1$$

□

A.3. Proof of Proposition 4.2

Proposition. Let's define two nonnegative weights

$$w_1(s, a) := \exp(\mathcal{T}V(s, a) - V(s)), \quad w_\tau(s, a) := \exp(\mathcal{T}V(s, a) - \tau V(s)),$$

where $\tau \in [0, 1]$. Consider weighted behavior cloning on a fixed dataset distribution d_{mix} :

$$\max_{\pi} \mathbb{E}_{(s,a) \sim d_{\text{mix}}} [w(s, a) \log \pi(a | s)].$$

Then the optimal policy for $w = w_1$ is also optimal for $w = w_\tau$ (and vice versa). In particular, both objectives share the same optimal solution $\pi^*(\cdot | s)$.

Proof. Let $\mu(a | s) := d_{\text{mix}}(s, a) / d_{\text{mix}}(s)$ be the empirical conditional action distribution of the dataset at state s . Since $d_{\text{mix}}(s)$ does not depend on π , the objective decomposes over states, and for each fixed s we solve

$$\max_{\pi(\cdot|s) \in \Delta(\mathcal{A})} \sum_a \mu(a | s) w(s, a) \log \pi(a | s). \quad (36)$$

The maximizer of (36) is given by a normalized reweighting:

$$\pi_w^*(a | s) = \frac{\mu(a | s) w(s, a)}{\sum_b \mu(b | s) w(s, b)}. \quad (37)$$

Now note that w_τ differs from w_1 by a factor depending only on s :

$$w_\tau(s, a) = \exp(\mathcal{T}V(s, a) - \tau V(s)) = \exp((1 - \tau)V(s)) \exp(\mathcal{T}V(s, a) - V(s)) = c_\tau(s) w_1(s, a),$$

where $c_\tau(s) := \exp((1 - \tau)V(s))$ is independent of a . Plugging into (37),

$$\pi_{w_\tau}^*(a | s) = \frac{\mu(a | s) c_\tau(s) w_1(s, a)}{\sum_b \mu(b | s) c_\tau(s) w_1(s, b)} = \frac{\mu(a | s) w_1(s, a)}{\sum_b \mu(b | s) w_1(s, b)} = \pi_{w_1}^*(a | s).$$

Thus, the two weighted BC objectives have exactly the same optimal solution $\pi^*(\cdot | s)$. $\square$

B. More discussion on difference between ReDICE and ReCOIL

Both ReDICE and ReCOIL (Sikchi et al., 2024) address coverage issues via mixture distribution, but ReCOIL’s objective differs from ours in an important way. Let $d^{\text{mix}}(s, a) = \beta d^E(s, a) + (1 - \beta)d^S(s, a)$. Consider the following three objectives.

Original IL objective.

$$\min_{d^\pi} D_f(d^\pi(s, a) \| d^E(s, a)). \quad (38)$$

ReCOIL objective.

$$\begin{aligned} \min_{\pi, d} D_f(\beta d(s, a) + (1 - \beta)d^S(s, a) \| d^{\text{mix}}(s, a)) \\ \text{s.t. } d(s, a) = (1 - \gamma)d_0(s)\pi(a|s) + \gamma \sum_{s', a'} d(s', a')\mathcal{P}(s|s', a')\pi(a|s). \end{aligned} \quad (39)$$

ReDICE objective.

$$\begin{aligned} \min_d \mathbb{E}_{d(s, a)} \left[\log \frac{d^{\text{mix}}(s, a)}{d^E(s, a)} \right] + D_{\text{KL}}(d(s, a) \| d^{\text{mix}}(s, a)) \\ \text{s.t. } \sum_a d(s, a) = (1 - \gamma)d_0(s) + \gamma \sum_{s', a'} d(s', a')\mathcal{P}(s|s', a'). \end{aligned} \quad (40)$$

The constraints in both (39) and (40) are Bellman flow constraints to ensure that $d(s, a)$ is a valid occupancy measure $d^\pi(s, a)$ in (38). From Eqs. (38)–(40), ReDICE has three key advantages over ReCOIL:

- **Preserves the original objective while relaxing coverage.** ReCOIL matches divergences between mixture distributions that only share the same optimum as (1), whereas ReDICE remains exactly equivalent to (1) under KL.
- **Avoids adversarial max–min optimization.** The dual formulation of ReCOIL solves a max–min problem over Q and π (Theorem 1 in (Sikchi et al., 2024)), which may induce extrapolation errors from out-of-distribution actions in offline settings (Kumar et al., 2020). ReDICE reduces to a minimization over V , avoiding this issue.
- **Simpler and more theory-aligned optimization.** To mitigate the OOD issues discussed above, ReCOIL relies on an in-sample surrogate objective to jointly learn $Q(s, a)$ and $V(s)$ (Eqs. (11)–(12) in (Sikchi et al., 2024)). This introduces an additional hyperparameter τ that must be tuned per task (Appendix F.1 of (Sikchi et al., 2024)). As a result, the practical implementation of ReCOIL creates a gap between theory and practice and increases tuning complexity. In contrast, ReDICE uses a single minimization over V , reducing tuning complexity.

C. Experiment Setup and Implementation Details

We implement our approach in PyTorch and run experiments on a multi-GPU cluster consisting of NVIDIA RTX 3090 GPUs. Each run evaluates five random seeds in parallel on shared hardware resources (one GPU, 32 CPU cores, and 128 GB RAM). All methods are trained for one million gradient steps and evaluated every 5,000 steps.

Environment	State Dim	Action Dim	Suboptimal Dataset	Dataset Details
Hopper	11	3	random+expert	1e6 random transitions + 200 expert trajectories
			medium+expert	1e6 medium transitions + 200 expert trajectories
			random+few-expert	1e6 random transitions + 30 expert trajectories
			medium+few-expert	1e6 medium transitions + 30 expert trajectories
Walker2d	17	6	random+expert	1e6 random transitions + 200 expert trajectories
			medium+expert	1e6 medium transitions + 200 expert trajectories
			random+few-expert	1e6 random transitions + 30 expert trajectories
			medium+few-expert	1e6 medium transitions + 30 expert trajectories
Halfcheetah	17	6	random+expert	1e6 random transitions + 200 expert trajectories
			medium+expert	1e6 medium transitions + 200 expert trajectories
			random+few-expert	1e6 random transitions + 30 expert trajectories
			medium+few-expert	1e6 medium transitions + 30 expert trajectories
Ant	27	8	random+expert	1e6 random transitions + 200 expert trajectories
			medium+expert	1e6 medium transitions + 200 expert trajectories
			random+few-expert	1e6 random transitions + 30 expert trajectories
			medium+few-expert	1e6 medium transitions + 30 expert trajectories
Pen	45	24	cloned+expert	5e6 cloned transitions + 30 expert trajectories
			human+expert	5000 human transitions + 30 expert trajectories
Door	39	28	cloned+expert	1e6 cloned transitions + 30 expert trajectories
			human+expert	6729 human transitions + 30 expert trajectories
Hammer	46	26	cloned+expert	1e6 cloned transitions + 30 expert trajectories
			human+expert	11310 human transitions + 30 expert trajectories
Kitchen	59	9	partial+expert	136950 partial transitions + 1 expert trajectory
			mixed+expert	136950 mixed transitions + 1 expert trajectory

Table 3. Overview of D4RL tasks and their respective suboptimal datasets used in the offline IL setting.

C.1. D4RL tasks

We adopt the same offline IL benchmark introduced in ReCOIL (Sikchi et al., 2024), which is constructed using datasets from the D4RL framework (Fu et al., 2020) and evaluated on MuJoCo control tasks. Across all environments, the expert dataset consists of a single expert trajectory. For locomotion tasks, suboptimal datasets—denoted as `random+expert`, `random+few-expert`, `medium+expert`, and `medium+few-expert` are created by combining expert demonstrations with lower-quality trajectories from the D4RL random-v2 and medium-v2 datasets. Specifically, the `random+expert` and `medium+expert` settings include 200 expert trajectories mixed with approximately one million transitions from the corresponding D4RL dataset, while the `x+few-expert` variants use only 30 expert trajectories. For manipulation tasks, each `x+expert` dataset is formed by mixing 30 expert trajectories with the full corresponding D4RL dataset, and we consistently use the -v0 versions of D4RL for these environments. An overview of all tasks is provided in Table 3.

C.2. ShipNaviSim task

The Maritime Navigation task was developed using historical data from a high-density traffic hotspot in the Singapore Strait, adhering to the AIS-driven simulation paradigm established in ShipNaviSim (Pham et al., 2025). We defined our planning region by selecting an area characterized by peak traffic volume and elevated collision risk, where numerous vessel paths intersect. Two years of AIS data were sourced from MarineTraffic (<https://www.marinetraffic.com/>), encompassing both static vessel attributes (dimensions, type, and ID) and dynamic movement variables (coordinates, speed, heading, and course-over-ground). Our analysis focuses on tankers and cargo ships, as their significant scale (200–300 meters) and limited maneuverability represent the highest navigational risk. Following a 10-second interpolation process,

the final dataset comprises 125,000 trajectories and 14 million environment transitions, with an average trajectory length of 100–150 steps. The data is divided into an 80% training set and a 20% evaluation set.

Observation Space The observation space is constructed from the view of the ego agent, providing the necessary context for multi-vessel interaction and decision-making. At each time step, the agent receives a historical sequence of its own state and the states of the 10 nearest vessels over a predefined window of past steps. Each historical data point includes the x and y coordinates, speed v , and heading h . The agent is also provided with its target goal location. To ensure a fair comparison across different algorithms, we utilized a consistent, straightforward neural network architecture. Rather than employing complex structural layers, the observation space (200-dimensional) is flattened and fed directly into the network.

Action Space We define the action space as a 3-dimensional continuous space representing changes in state, denoted as $\langle d_x, d_y, d_h \rangle$. These components correspond to the change in x and y coordinates and the change in heading h , respectively. The vessel’s velocity for the following step is calculated based on the displacement over a fixed time interval ($\delta_T = 10$ seconds), where $v_{t+1} = \sqrt{d_x^2 + d_y^2} / \delta_T$. This *delta action space* formulation (Gulino et al., 2023) is applicable to various mobile agents. By representing actions as the difference between consecutive states, we can utilize a simple Inverse Kinematics Model to derive the necessary actions from the state-only trajectories present in our historical dataset.

Evaluation Metrics To assess the performance of the navigation policies against human expert behavior, we adopt the *vessel-specific metrics* established in ShipNaviSim:

- **Goal-Conditioned ADE (GC-ADE):** This metric quantifies the average Euclidean distance between the trajectory generated by the learned policy and the ground-truth historical trajectory. For a modeled trajectory τ_m and a predicted trajectory τ_p , the error is averaged over the duration of the shorter sequence:

$$\text{GC-ADE} = \frac{1}{\min(T_m, T_p)} \sqrt{\sum_{t=1}^{\min(T_m, T_p)} (x_t^m - x_t^p)^2 + (y_t^m - y_t^p)^2}$$

- **Goal Rate:** This represents the success frequency of the ego agent in reaching its destination. A trial is considered successful if the agent comes within a 200-meter radius of the goal.
- **Near Miss Count:** This measures the average number of time-steps per episode where the ego agent is within 3 cable lengths (555 meters) of another vessel. While domain experts categorize this proximity as a "near-miss," we use this metric as a broader indicator of traffic density and potential navigational risk, rather than an absolute measure of imminent collision.
- **Max Curvature:** Curvature measures the angle change in the vessel’s course over ground at each time step. It indicates the sharpness of a turn and helps assess the maneuverability and alignment with expected vessel behavior, which cannot make sharp turns. By monitoring maximum curvature, we can ensure that the generated trajectories adhere to the physical motion constraints of real-world maritime vessels and avoid erratic or unrealistic maneuvers.

C.3. Hyperparameters

Our implementation is mainly based on the official codebases of ODICE (Mao et al., 2024) and SMODICE (Ma et al., 2022), adopting their core network configurations and parameters as detailed in Table 4. To enhance training stability, we follow (Garg et al., 2023), incorporating Layer Normalization (Ba et al., 2016) into the critic network. We set a fixed mixture ratio $\beta = 0.5$ for all experiments. For the parameter τ , we find that $\tau = 0.9$ works for most tasks except the Kitchen and human+expert tasks where $\tau = 1.0$ yields better results. We also note that other SOTA offline IL methods such as ReCOIL (Sikchi et al., 2024) and SMODICE (Ma et al., 2022), also adjust their hyperparameters for different sets of tasks. We follow such tuned hyperparameters as reported in the respective papers for different baselines for fair comparisons.

Table 4. Hyperparameters of ReDICE

Type	Hyperparameter	Value
Discriminator	Network Size	[256, 256]
	Activation Function	Tanh
	Learning Rate	3e-4
	Training Length	5000
	Batch Size	256
	Optimizer	Adam
Actor	Network Size	[1024, 1024]
	Activation Function	ReLU
	Learning Rate	3e-4
	Weight Decay	1e-3
	Training Length	1M steps
	Batch Size	512
	Optimizer	Adam
	Dropout Rate	0.1
Critic	LR decay schedule	cosine
	Network Size	[256, 256]
	Activation Function	ReLU
	Learning Rate	3e-4
	Weight Decay	5e-6
	Training Length	1M steps
	Batch Size	512
	Optimizer	Adam
	Soft Target Update Rate	0.005
	Discount Factor γ	0.99

D. Detailed Ablation Results

Revisiting Distribution Correction for Offline Imitation Learning with Suboptimal Dataset

Table 5. Average normalized return over last 10 evaluations of ReDICE against its variants on the D4RL suboptimal datasets with 1 expert trajectory. The mean and std are obtained over 5 random seeds. Offline IL methods with avg. perf within the std-dev of the top performing approach is in **bold**.

Suboptimal Dataset	Env	ReDICE w/o mixture distribution	ReDICE w/o Gumbel loss	ReDICE w/o τ	ReDICE	Expert
random+ expert	hopper	0.93 $\pm$ 0.02	6.79 $\pm$ 2.80	52.09 $\pm$ 2.81	109.80 $\pm$ 0.31	111.3
	halfcheetah	2.25 $\pm$ 0.00	2.24 $\pm$ 0.01	23.88 $\pm$ 1.65	91.82 $\pm$ 0.44	88.83
	walker2d	-0.14 $\pm$ 0.03	3.59 $\pm$ 3.95	73.97 $\pm$ 14.88	107.98 $\pm$ 0.12	106.92
	ant	27.41 $\pm$ 0.29	53.09 $\pm$ 5.52	82.22 $\pm$ 4.33	126.32 $\pm$ 0.81	130.75
random+ few-expert	hopper	1.04 $\pm$ 0.13	2.92 $\pm$ 1.00	29.58 $\pm$ 0.41	93.63 $\pm$ 1.69	111.33
	halfcheetah	2.25 $\pm$ 0.00	2.23 $\pm$ 0.00	20.12 $\pm$ 1.24	79.31 $\pm$ 4.59	88.83
	walker2d	-0.17 $\pm$ 0.06	1.54 $\pm$ 1.39	7.37 $\pm$ 1.02	108.26 $\pm$ 0.03	106.92
	ant	30.53 $\pm$ 1.93	33.24 $\pm$ 1.43	47.32 $\pm$ 5.95	108.15 $\pm$ 3.59	130.75
medium+ expert	hopper	0.71 $\pm$ 0.04	53.85 $\pm$ 0.61	93.74 $\pm$ 3.28	111.01 $\pm$ 0.38	111.33
	halfcheetah	2.25 $\pm$ 0.00	42.74 $\pm$ 0.18	76.83 $\pm$ 2.79	91.80 $\pm$ 0.42	88.83
	walker2d	-0.21 $\pm$ 0.08	72.75 $\pm$ 1.27	108.90 $\pm$ 0.20	108.40 $\pm$ 0.50	106.92
	ant	1.20 $\pm$ 12.79	93.43 $\pm$ 2.58	116.03 $\pm$ 6.37	123.24 $\pm$ 1.48	130.75
medium few-expert	hopper	9.14 $\pm$ 14.53	54.06 $\pm$ 0.47	38.41 $\pm$ 6.53	109.50 $\pm$ 1.23	111.33
	halfcheetah	2.25 $\pm$ 0.00	42.39 $\pm$ 0.14	53.44 $\pm$ 1.02	81.57 $\pm$ 2.94	88.83
	walker2d	0.00 $\pm$ 0.25	69.03 $\pm$ 2.36	75.25 $\pm$ 16.88	108.70 $\pm$ 0.44	106.92
	ant	-11.24 $\pm$ 9.95	88.57 $\pm$ 3.51	109.66 $\pm$ 4.20	111.73 $\pm$ 2.37	130.75
cloned+expert	pen	31.21 $\pm$ 3.61	20.57 $\pm$ 16.71	61.78 $\pm$ 10.00	88.84 $\pm$ 8.84	106.42
	door	-0.00 $\pm$ 0.01	0.38 $\pm$ 0.66	44.29 $\pm$ 35.07	92.23 $\pm$ 2.45	103.94
	hammer	0.31 $\pm$ 0.01	0.29 $\pm$ 0.01	43.19 $\pm$ 16.88	104.86 $\pm$ 10.07	125.71
Average		5.25	33.88	60.95	103.01	109.86

Table 6. Average normalized return over last 10 evaluations of ReDICE with different τ values on the D4RL suboptimal datasets with 1 expert trajectory. The mean and std are obtained over 5 random seeds. Offline IL methods with avg. perf within the std-dev of the top performing approach is in **bold**.

Suboptimal Dataset	Env	$\tau = 1.0$	$\tau = 0.9$	$\tau = 0.7$	$\tau = 0.5$	$\tau = 0.3$	$\tau = 0.1$	Expert
random+ expert	hopper	52.09 $\pm$ 2.81	109.80 $\pm$ 0.31	109.75 $\pm$ 1.07	26.95 $\pm$ 12.75	10.64 $\pm$ 2.84	12.17 $\pm$ 4.30	111.3
	halfcheetah	23.88 $\pm$ 1.65	91.82 $\pm$ 0.44	92.02 $\pm$ 0.05	91.72 $\pm$ 0.13	91.32 $\pm$ 0.75	90.84 $\pm$ 0.79	88.83
	walker2d	73.97 $\pm$ 14.88	107.98 $\pm$ 0.12	108.25 $\pm$ 0.19	103.57 $\pm$ 7.78	49.26 $\pm$ 16.42	22.45 $\pm$ 19.59	106.92
	ant	82.22 $\pm$ 4.33	126.32 $\pm$ 0.81	114.33 $\pm$ 2.29	78.93 $\pm$ 8.83	65.70 $\pm$ 4.73	59.54 $\pm$ 3.46	130.75
random+ few-expert	hopper	29.58 $\pm$ 0.41	93.63 $\pm$ 1.69	6.72 $\pm$ 1.03	5.95 $\pm$ 0.71	7.02 $\pm$ 0.98	6.43 $\pm$ 0.94	111.33
	halfcheetah	20.12 $\pm$ 1.24	79.31 $\pm$ 4.59	74.74 $\pm$ 2.26	69.62 $\pm$ 6.59	64.01 $\pm$ 7.41	61.79 $\pm$ 9.49	88.83
	walker2d	7.37 $\pm$ 1.02	108.26 $\pm$ 0.03	108.09 $\pm$ 0.61	53.47 $\pm$ 8.50	5.99 $\pm$ 3.30	4.66 $\pm$ 2.80	106.92
	ant	47.32 $\pm$ 5.95	108.15 $\pm$ 3.59	58.83 $\pm$ 6.68	29.16 $\pm$ 1.18	27.89 $\pm$ 2.43	27.59 $\pm$ 1.58	130.75
medium+ expert	hopper	93.74 $\pm$ 3.28	111.01 $\pm$ 0.38	110.02 $\pm$ 0.29	110.55 $\pm$ 0.03	110.24 $\pm$ 0.23	109.81 $\pm$ 0.94	111.33
	halfcheetah	76.83 $\pm$ 2.79	91.80 $\pm$ 0.42	89.98 $\pm$ 0.29	55.92 $\pm$ 1.50	43.79 $\pm$ 0.38	42.60 $\pm$ 0.20	88.83
	walker2d	108.90 $\pm$ 0.20	108.40 $\pm$ 0.50	108.06 $\pm$ 0.02	108.15 $\pm$ 0.13	107.79 $\pm$ 0.76	107.42 $\pm$ 0.41	106.92
	ant	116.03 $\pm$ 6.37	123.24 $\pm$ 1.48	108.85 $\pm$ 4.98	98.13 $\pm$ 4.33	93.97 $\pm$ 4.44	94.17 $\pm$ 2.93	130.75
medium few-expert	hopper	38.41 $\pm$ 6.53	109.50 $\pm$ 1.23	109.87 $\pm$ 1.37	108.69 $\pm$ 0.57	102.66 $\pm$ 7.28	107.36 $\pm$ 2.39	111.33
	halfcheetah	53.44 $\pm$ 1.02	81.57 $\pm$ 2.94	46.10 $\pm$ 1.57	43.53 $\pm$ 0.65	42.67 $\pm$ 0.35	42.61 $\pm$ 0.29	88.83
	walker2d	75.25 $\pm$ 16.88	108.70 $\pm$ 0.44	106.74 $\pm$ 0.80	106.27 $\pm$ 0.03	104.21 $\pm$ 1.92	58.77 $\pm$ 6.18	106.92
	ant	109.66 $\pm$ 4.20	111.73 $\pm$ 2.37	93.30 $\pm$ 0.41	92.07 $\pm$ 3.43	87.74 $\pm$ 4.17	84.99 $\pm$ 2.74	130.75
cloned+expert	pen	61.78 $\pm$ 10.00	88.84 $\pm$ 8.84	66.91 $\pm$ 6.94	73.45 $\pm$ 7.47	67.32 $\pm$ 5.54	59.04 $\pm$ 6.99	106.42
	door	44.29 $\pm$ 35.07	92.23 $\pm$ 2.45	89.98 $\pm$ 18.85	91.94 $\pm$ 10.27	27.67 $\pm$ 47.87	25.15 $\pm$ 43.65	103.94
	hammer	43.19 $\pm$ 16.88	104.86 $\pm$ 10.07	83.83 $\pm$ 36.16	85.82 $\pm$ 37.01	45.77 $\pm$ 50.51	42.99 $\pm$ 56.75	125.71
Average		60.95	103.01	88.76	75.47	60.82	55.81	109.86

Table 7. Average normalized return over last 10 evaluations of ReDICE with different β values on the D4RL suboptimal datasets with 1 expert trajectory. The mean and std are obtained over 5 random seeds. Offline IL methods with avg. perf within the std-dev of the top performing approach is in **bold**.

Suboptimal Dataset	Env	$\beta = 0.1$	$\beta = 0.3$	$\beta = 0.5$	$\beta = 0.7$	$\beta = 0.9$	Expert
random+ expert	hopper	105.94 $\pm$ 4.96	109.97 $\pm$ 0.49	109.80 $\pm$ 0.31	102.21 $\pm$ 2.53	65.84 $\pm$ 1.69	111.3
	halfcheetah	89.26 $\pm$ 0.09	91.20 $\pm$ 0.37	91.82 $\pm$ 0.44	91.26 $\pm$ 0.50	81.60 $\pm$ 0.20	88.83
	walker2d	108.13 $\pm$ 0.23	108.19 $\pm$ 0.17	107.98 $\pm$ 0.12	107.96 $\pm$ 0.50	86.77 $\pm$ 3.08	106.92
	ant	69.52 $\pm$ 4.44	68.28 $\pm$ 9.99	126.32 $\pm$ 0.81	102.60 $\pm$ 18.49	120.33 $\pm$ 4.19	130.75
random+ few-expert	hopper	6.20 $\pm$ 0.55	88.31 $\pm$ 3.62	93.63 $\pm$ 1.69	80.43 $\pm$ 6.40	75.28 $\pm$ 19.26	111.33
	halfcheetah	2.32 $\pm$ 0.04	56.46 $\pm$ 3.33	79.31 $\pm$ 4.59	82.75 $\pm$ 1.17	59.03 $\pm$ 26.80	88.83
	walker2d	2.68 $\pm$ 0.27	107.75 $\pm$ 0.68	108.26 $\pm$ 0.03	108.31 $\pm$ 0.20	88.64 $\pm$ 20.27	106.92
	ant	118.82 $\pm$ 1.57	118.19 $\pm$ 1.88	108.15 $\pm$ 3.59	105.52 $\pm$ 16.85	91.34 $\pm$ 3.23	130.75
medium+ expert	hopper	110.00 $\pm$ 1.67	110.67 $\pm$ 0.50	111.01 $\pm$ 0.38	60.00 $\pm$ 46.69	32.35 $\pm$ 8.38	111.33
	halfcheetah	42.45 $\pm$ 0.19	42.51 $\pm$ 0.07	91.80 $\pm$ 0.42	63.33 $\pm$ 16.82	7.12 $\pm$ 2.01	88.83
	walker2d	107.48 $\pm$ 0.69	108.08 $\pm$ 0.05	108.40 $\pm$ 0.50	108.37 $\pm$ 0.11	8.32 $\pm$ 1.65	106.92
	ant	90.88 $\pm$ 2.59	89.27 $\pm$ 1.31	123.24 $\pm$ 1.48	94.92 $\pm$ 2.96	75.76 $\pm$ 3.49	130.75
medium few-expert	hopper	102.84 $\pm$ 5.21	108.49 $\pm$ 1.49	109.50 $\pm$ 1.23	27.07 $\pm$ 14.61	15.00 $\pm$ 2.60	111.33
	halfcheetah	81.17 $\pm$ 6.46	83.06 $\pm$ 2.95	81.57 $\pm$ 2.94	85.37 $\pm$ 3.87	58.88 $\pm$ 5.56	88.83
	walker2d	80.84 $\pm$ 2.58	104.84 $\pm$ 2.61	108.70 $\pm$ 0.44	81.81 $\pm$ 22.97	80.22 $\pm$ 20.55	106.92
	ant	88.74 $\pm$ 3.67	88.11 $\pm$ 0.65	111.73 $\pm$ 2.37	88.58 $\pm$ 2.47	91.10 $\pm$ 4.46	130.75
Average		75.45	92.71	104.45	86.91	64.85	109.46

Table 8. Performance comparison of SMODICE variants on D4RL suboptimal datasets with 1 expert trajectories. The mean and std are obtained over multiple random seeds. Methods with average performance within one standard deviation of the top-performing approach are shown in **bold**.

Suboptimal Dataset	Env	SMODICE	SMODICE w/ orthogonal	SMODICE w/ all
random+ expert	hopper	104.09 $\pm$ 1.82	5.79 $\pm$ 0.30	0.93 $\pm$ 0.02
	halfcheetah	80.92 $\pm$ 1.94	2.25 $\pm$ 0.00	2.25 $\pm$ 0.00
	walker2d	107.46 $\pm$ 0.22	1.29 $\pm$ 0.07	-0.14 $\pm$ 0.03
	ant	126.48 $\pm$ 1.78	38.66 $\pm$ 9.82	27.41 $\pm$ 0.29
random+ few-expert	hopper	58.72 $\pm$ 7.09	5.34 $\pm$ 0.07	1.04 $\pm$ 0.13
	halfcheetah	4.16 $\pm$ 2.83	2.25 $\pm$ 0.00	2.25 $\pm$ 0.00
	walker2d	3.48 $\pm$ 2.57	1.40 $\pm$ 0.03	-0.17 $\pm$ 0.06
	ant	13.26 $\pm$ 5.01	29.50 $\pm$ 0.32	30.53 $\pm$ 1.93
medium+ expert	hopper	57.83 $\pm$ 3.52	52.89 $\pm$ 0.25	0.71 $\pm$ 0.04
	halfcheetah	66.52 $\pm$ 2.32	42.67 $\pm$ 0.08	2.25 $\pm$ 0.00
	walker2d	39.71 $\pm$ 10.78	72.71 $\pm$ 1.08	-0.21 $\pm$ 0.08
	ant	109.73 $\pm$ 1.88	99.40 $\pm$ 3.99	1.20 $\pm$ 12.79
medium+ few-expert	hopper	53.33 $\pm$ 0.80	52.83 $\pm$ 0.98	9.14 $\pm$ 14.53
	halfcheetah	41.47 $\pm$ 1.99	42.38 $\pm$ 0.29	2.25 $\pm$ 0.00
	walker2d	13.78 $\pm$ 14.21	71.75 $\pm$ 1.62	0.00 $\pm$ 0.25
	ant	89.22 $\pm$ 2.44	89.90 $\pm$ 0.59	-11.24 $\pm$ 9.95
cloned+expert	pen	33.97 $\pm$ 19.99	22.25 $\pm$ 2.44	31.21 $\pm$ 3.61
	door	3.57 $\pm$ 3.38	0.01 $\pm$ 0.00	-0.00 $\pm$ 0.01
	hammer	3.98 $\pm$ 5.45	1.09 $\pm$ 0.56	0.31 $\pm$ 0.01
Average		53.25	33.39	5.25

Table 9. Performance comparison of ReCOIL variants and ReDICE on D4RL suboptimal datasets with 1 expert trajectory. The reported values are normalized returns averaged over multiple random seeds. The best-performing method for each task is shown in **bold**. We evaluate ReCOIL with τ -scaling by replacing its policy extraction and tuning $\tau \in \{0.5, 0.7, 0.9, 1.0\}$. We observe consistent trends with ReDICE: for ReCOIL, $\tau = 0.9$ also performs best on locomotion and cloned tasks, while $\tau = 1.0$ works better on Kitchen and human datasets.

Dataset	Env	ReCOIL w/ τ -scaling	ReCOIL	ReDICE
random+expert	hopper	102.79	98.33	109.80
	halfcheetah	90.94	78.78	91.82
	walker2d	71.79	105.72	107.98
	ant	126.07	116.43	126.32
random+few-expert	hopper	90.15	86.65	93.63
	halfcheetah	37.44	5.25	79.31
	walker2d	34.05	12.59	108.26
	ant	96.71	56.68	108.15
medium+expert	hopper	110.84	85.25	111.01
	halfcheetah	75.91	78.88	91.80
	walker2d	108.40	108.27	108.40
	ant	123.02	119.21	123.24
medium+few-expert	hopper	92.59	38.88	109.50
	halfcheetah	57.85	41.90	81.57
	walker2d	107.86	71.44	108.70
	ant	108.79	104.03	111.73
cloned+expert	pen	40.80	45.59	88.84
	door	87.61	16.18	92.23
	hammer	31.65	2.60	104.86
human+expert	pen	84.35	85.04	82.62
	door	92.36	84.02	94.31
	hammer	80.41	86.45	107.97
partial+expert	kitchen	54.25	48.80	54.00
mixed+expert	kitchen	49.17	49.35	48.25
Average		81.49	67.76	97.68